ASSISTANT • 架构说明

文件搜索：模块划分与连接

现有代码里有哪些模块 · 它们靠什么连起来 · 哪里没连上

基于 v0.3.0 + A1 落地后的代码现状

一页结论

· 模块切分是清楚的：七层、职责单一

· 连接是不完整的：一半注入、一半静态直调

· 缺口集中在「编排层 → 采集层」这一跳

· 这正是「能跑，但改不动、测不了」的根源

OVERVIEW

一页看懂

模块划分

七层，职责单一

- 触发 · 界面 · 编排 · 匹配
- 采集 · 持久化 · 基础
- 每一层都有明确的输入输出
- 没有循环依赖

连接方式

三种并存

- 信号 / 槽：跨窗口与触发
- setter 注入：数据库类依赖
- 编译期静态调用：算法与采集
- 第三种正是问题所在

连接缺口

两处没接上

- 历史库只写不读，从未反馈到排序
- 采集层无注入点，实现不可替换
- 全局静态索引无法 mock
- 直接决定 A2 的改造范围

一句话结论：搜索链路的每一步都能单独说清楚，模块边界也不含糊；但只要「谁拥有谁、谁能替换谁」没定下来，加功能就会变成改 AppBridge —— 所以下一步的重点不是加模块，而是把连接方式统一成可替换的那一种。

模块划分

七层模块：按职责切，不按功能切

层	模块	所在文件	职责
① 触发层	HotkeyManager · TrayManager	core/HotkeyManager · ui/TrayManager	全局热键注册；托盘图标与菜单
② 界面层	MainWindow · MainPanel · Style	resources/qml/*.qml	搜索浮窗、主面板；主题单例
③ 编排层	AppBridge · PanelBridge	ui/AppBridge · ui/PanelBridge	搜索编排与执行分发；面板 CRUD
④ 匹配层	FuzzyMatcher · Calculator	core/FuzzyMatcher · core/Calculator	五级模糊打分；算式求值
⑤ 采集层	AppIndexer · FileSearcher	core/AppIndexer · core/FileSearcher	Start Menu 应用；文件系统遍历
⑥ 持久化层	HistoryDatabase · Todo/Notes/Clipboard	database/*.cpp（4 个独立连接）	搜索历史；三个功能各自的库
⑦ 基础层	Logger · ConfigManager	core/Logger · core/ConfigManager	双 sink 日志；JSON 配置与持久化

分层依据是「谁会调谁」，不是功能分类：④⑤两层都是无状态算法/采集，可被③层任意调用。

两个采集器：同一个位置，完全不同的做法

维度	AppIndexer（应用）	FileSearcher（文件）
采集对象	用户 + 全局 Start Menu 的 .lnk	6 个标准用户目录下的文件
实现手段	IShellLink COM 解析快捷方式目标	自控前序 DFS（walkDir 递归）
接口形态	scan() → QList<Entry>	命名空间 + 文件级静态 s_index
调用时机	main.cpp:68 启动时一次	main.cpp:72 启动时一次
规模上限	无上限（实际 31 条）	硬上限 20000 条
失败处理	解析不出目标就静默跳过	目录级剪枝；不跟随 junction
实测开销	133 ms	888 ms（触顶截断）

同为「采集层」，一个返回集合、一个持有全局状态 —— 这个差异决定了它们后面能不能被替换。

一个共享叶子，两个互不相识的调用方

FuzzyMatcher 的五级打分（分数即优先级）

1000	900	800	600-	400-
精确	前缀	词边界	包含·位置加权	编辑距离

关键连接事实：它是被两条链路共用的叶子依赖

调用方 A AppBridge.cpp:92 → FuzzyMatcher::score(query, item.title) （打分对象：内置命令与应用名）

调用方 B FileSearcher.cpp:148 → FuzzyMatcher::score(query, e.name) （打分对象：文件名）

影响 改写打分逻辑会同时改变两条链路的结果；计划中的「小写形式预计算」必须两边一起改，否则排序出现分叉。

Calculator 也挂在同一层：AppBridge.cpp:143，门禁为「含数字 + 含运算符 + 长度≥3」。

AppBridge::search：四步，一个排序

01 空查询短路

query 为空 → 直接返回 10 条内置命令

此时不会触碰文件搜索

保证空列表不会爆炸

02 内置与应用打分

遍历 m_allItems（内置 10 + 应用 31）

逐条 FuzzyMatcher::score

保留 score > 0 的命中

03 追加文件结果

FileSearcher::search(query, 20)

文件不进入 m_allItems，只在搜索时追加

priority=1，同分时排在后面

04 整体排序 + 计算器

拼成一个大 list 后 std::sort

score 降序 → priority 降序

命中算式则 prepend score=1001

副作用：只记 m_lastQuery；A1 之后不再在 search() 里写库，**真正的历史写入挪到了 executeCommand()。**四类结果拼成一个大列表再全量排序，是 A7 要换成多路归并的地方。

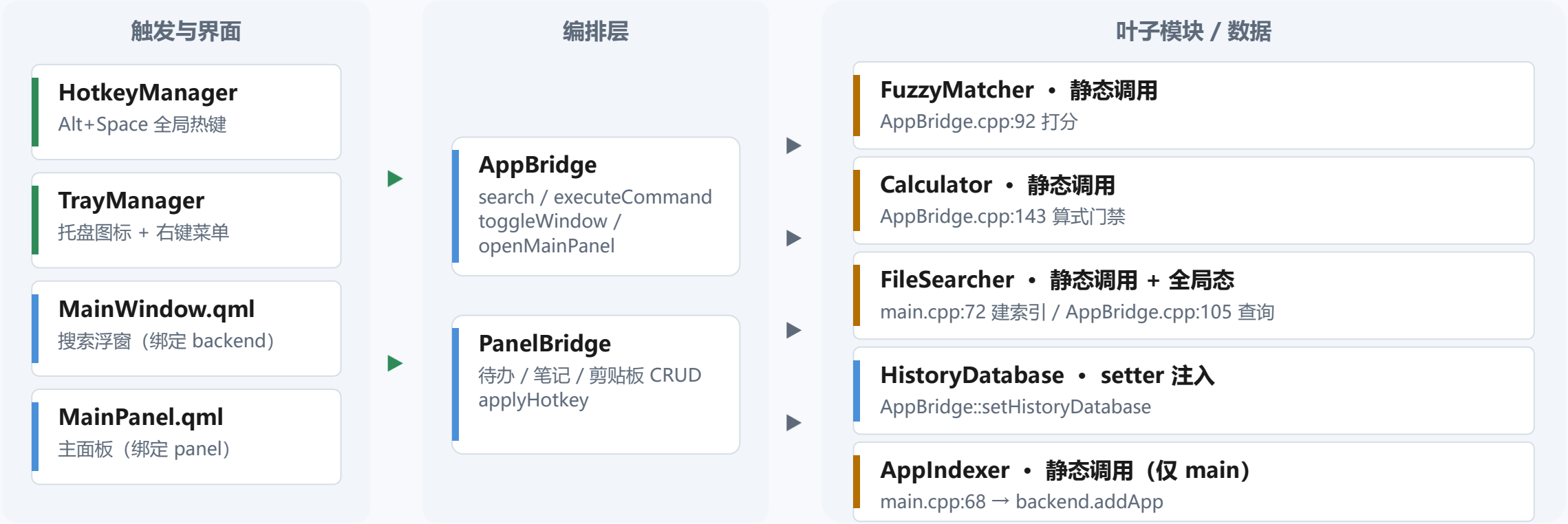
executeCommand：按结果类型四分派

分支	判据	动作	写入的历史
计算器	type == Calculator	结果复制到剪贴板（去掉 "= " 前缀）	search_history
应用	id 以 app:// 开头	openUrl，失败再退 QProcess::startDetached	+ app_launch_history
文件	type == File	openUrl，交给系统默认程序打开	search_history
内置命令	其余	executeAction 按 id 字符串逐个比对分发	+ command_history

连接要点：执行链路是 search 的镜像 —— 也是四条分支、也由 AppBridge 独占。AppBridge 同时承担「编排 + 打分 + 分发 + 文案」四种职责，是当前最接近上帝对象的模块（347 行，C++ 模块中最大）。

连接 ①

组合根：所有接线集中在 main.cpp



另有 2 个 lambda 连接：visibleChanged → 存窗口坐标；mainPanelRequested → 显示面板并隐藏浮窗。

组合根的存在本身是好事：依赖关系集中可见。问题在于第三步「静态调用」绕过了它。

连接 ②

两个窗口、两个引擎、两个桥

	搜索浮窗	主面板
入口 QML	qrc:/qml/MainWindow.qml	qrc:/qml/MainPanel.qml
加载者	main.cpp:87 engine	main.cpp:140 panelEngine
注入的桥	backend = AppBridge	panel = PanelBridge
共同注入	config · appVersion	config · appVersion
初始可见性	绑定 backend.visible（初值 false）	visible: false，由 🏠 / 托盘触发
共享主题	Style 单例（读 config.theme）	Style 单例（读 config.theme）

隐含前提：两个 engine 各自独立，但 Style 单例会在每个 engine 里各实例化一份，而它的 isDark 依赖 config.theme —— 也就是说「每个 engine 都必须注入 config」是个未写进文档的硬约束，漏掉一个不会报错，只会静默取不到主题。

两套桥换来的是互不干扰（搜索浮窗不受面板影响）；代价是主题单例与缓存各一份。

连接 ③

依赖注入只做了一半

依赖关系	连接方式	可替换 / 可 mock
AppBridge → HistoryDatabase	setter 注入（setHistoryDatabase）	✅ 可以
PanelBridge → Todo / Notes / Clipboard	setter 注入（三个 setter）	✅ 可以
PanelBridge → Config / HotkeyManager	setter 注入	✅ 可以
AppBridge → FileSearcher	编译期静态调用 + 全局静态索引	❌ 不行
AppBridge → FuzzyMatcher / Calculator	编译期静态调用（namespace）	❌ 不行
main → AppIndexer	编译期静态调用	❌ 不行

这个分裂带来的直接后果：数据库类可以随时换实现（测试时给个假的就行），而算法与采集类一旦被调用就写死 —— 所以 A2 想加「legacy / v2」回退开关时，会发现 FileSearcher 根本没有挂载点，必须先立一个门面（FileIndex）出来。

连接缺口

两处没接上，一处接错了位置

缺口 ①

历史库只写不读

HistoryDatabase 有 3 个读取接口：

recentSearches / frequentApps / launchCount

——全仓库零调用方（已实测）。

databaseError 信号发出但无人 connect。

→ 历史在积累，却从未反馈到排序。

缺口 ②

采集层没有注入点

FileSearcher 是 namespace + 文件级

静态 s_index：无实例、无构造、无接口。

AppBridge 只能直接写 FileSearcher::search()。

→ 换实现要改 AppBridge，回退开关没处挂。

缺口 ③

采集阻塞在窗口之前

main.cpp:72 buildIndex() 同步执行，

engine.load() 在 87 行 —— 阻塞在加载之前。

实测 888 ms（占启动到热键就绪的 49%）。

热键 123 行才注册，app.exec() 在 158 行。

→ 启动后约 1.8 s 内按 Alt+Space 无反应。

①②属于架构债，③是它俩的运行时表现：采集层既不可替换，又被放在了错误的时机。

连接的重构方向

现状 → A2：把「静态调用」换成「注入」

方面	现状（v0.3.0 + A1）	A2 目标
采集层形态	namespace + 文件级静态 s_index	FileIndex 门面 + FileScanner 后台线程
索引发布	无机制，建完即冻结	SnapshotManager 以 COW 方式发布快照
查询路径	每次按键线性扫全量索引（2 万条）	QueryEngine 独立成层，查只读快照
编排层依赖	AppBridge 直接写 FileSearcher::search()	注入 FileIndex*，与 setHistoryDatabase 同风格
回退能力	无（旧实现是唯一实现）	fileIndex.engine = legacy | v2，一行配置回退

判据：改造是否成功，不看新增了多少文件，而看 AppBridge 能不能在不改一行代码的前提下换掉文件搜索实现 —— 能换，连接方式才算真的统一了。

SUMMARY

小结

划分

七层，边界清楚

触发 / 界面 / 编排 / 匹配 / 采集 / 持久化 / 基础

连接

三种方式并存

信号槽 ✅ · setter 注入 ✅ · 编译期静态调用 ⚠

缺口

编排层 → 采集层这一跳

无注入点 + 无实例 + 全局静态索引，占启动阻塞的 49%

方向

统一成「可替换」那一种

A2 立 FileIndex 门面，legacy / v2 一行配置回退